SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL - SENAI
CURSO DE DESENVOLVIMENTO MOBILE FLUTTER
DISCIPLINA: LÓGICA DE PROGRAMAÇÃO
PROFESSOR: NAIRO ANDRE JESUS SANCHES
ALUNO: ALEX CRISTHIAN LIMA DE MEDEIROS

SISTEMA DE CHAMADOS INTELIGENTES EM TI

Relatório e Implementação em JavaScript

MOSSORÓ - RN
2026

1. Contextualização da atividade e decisões

A atividade consiste no desenvolvimento de um sistema de triagem de chamados de suporte técnico, responsável por classificar os problemas de acordo com seus níveis de impacto e urgência.

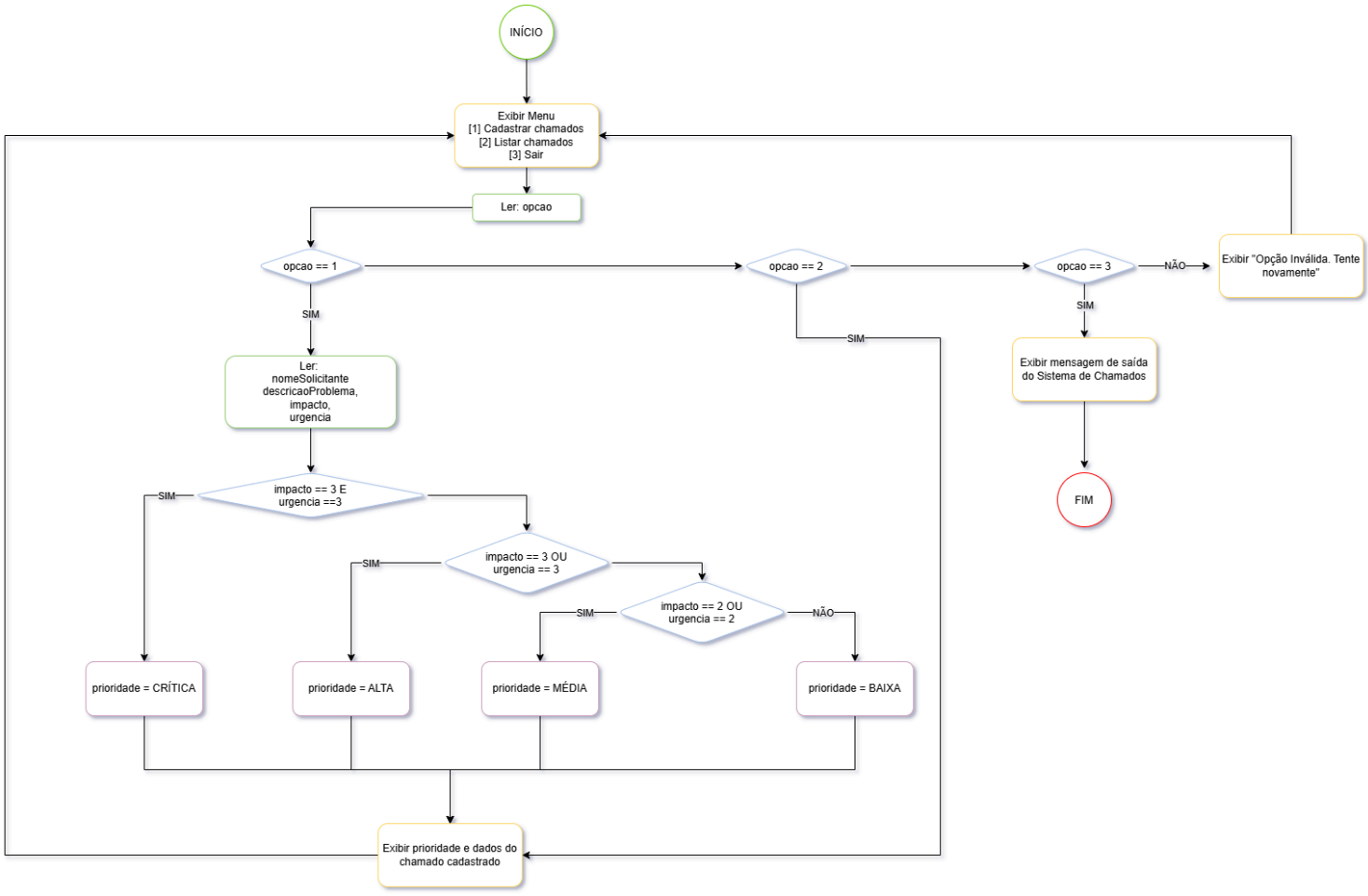
Como primeira etapa, foi elaborado um fluxograma para modelar o problema e organizar visualmente a sequência de operações, identificando as entradas, decisões e repetições necessárias. Em seguida, foi realizada a implementação em JavaScript, utilizando Node.js, e, por último, foi desenvolvido o pseudocódigo para representar a lógica da solução de forma estruturada.

Durante o planejamento, optei por criar um sistema com interação direta com o usuário, por meio de um menu que permite cadastrar e listar chamados. Também decidi que a execução deveria ser contínua, possibilitando realizar várias operações sem reiniciar o programa, até que o usuário escolha sair.

Para a implementação, utilizei funções para separar as principais responsabilidades do sistema e um vetor para armazenar os chamados durante a execução. Também foram incluídas validações para garantir que os níveis de impacto e urgência estejam entre 1 e 3, assegurando que a classificação siga as regras propostas na atividade.

2. Fluxograma

O fluxograma representa o funcionamento do sistema de triagem de chamados, desde a exibição do menu inicial até a execução da opção escolhida pelo usuário. Caso seja selecionado o cadastro, são solicitados os dados do chamado e realizada a classificação com base nos níveis de impacto e urgência, definindo uma das quatro prioridades: crítica, alta, média ou baixa. Em seguida, os dados e a prioridade são exibidos. O fluxograma também contempla a listagem dos chamados cadastrados, o tratamento de opções inválidas e a saída do sistema. Ao final de cada operação, o fluxo retorna ao menu principal, permitindo que o usuário continue utilizando o sistema até escolher a opção de encerramento.



3. Pseudocódigo

O pseudocódigo desenvolvido leva em consideração a implementação realizada em JavaScript

```
INÍCIO
  CRIAR lista chamados vazia
  FUNÇÃO mostrarMenu()
    ESCREVER "Sistema de Triagem de Chamados"
    ESCREVER "1 - Cadastrar novo chamado"
    ESCREVER "2 - Listar chamados cadastrados"
    ESCREVER "3 - Sair do sistema"
  FIM FUNÇÃO

  FUNÇÃO classificarPrioridade(impacto, urgencia)
    SE impacto = 3 E urgencia = 3 ENTÃO
      prioridade <- "CRÍTICA"
    SENÃO SE impacto = 3 OU urgencia = 3 ENTÃO
      prioridade <- "ALTA"
    SENÃO SE impacto = 2 OU urgencia = 2 ENTÃO
      prioridade <- "MÉDIA"
    SENÃO
      prioridade <- "BAIXA"
    FIM SE
    RETORNAR prioridade
  FIM FUNÇÃO

  FUNÇÃO cadastrarChamado()
    ESCREVER "Digite o nome do solicitante:"
    LER nomeSolicitante
    ESCREVER "Digite a descrição do problema:"
    LER descricaoProblema
    ESCREVER "Digite o impacto (1 - Baixo, 2 - Médio, 3 - Alto):"
    LER impacto
    ENQUANTO impacto NÃO FOR 1, 2 OU 3 FAÇA
      ESCREVER "Valor inválido! Digite 1, 2 ou 3."
      LER impacto
    FIM ENQUANTO
    ESCREVER "Digite a urgência (1 - Baixa, 2 - Média, 3 - Alta):"
    LER urgencia
    ENQUANTO urgencia NÃO FOR 1, 2 OU 3 FAÇA
      ESCREVER "Valor inválido! Digite 1, 2 ou 3."
      LER urgencia
    FIM ENQUANTO

    prioridade <- classificarPrioridade(impacto, urgencia)

    CRIAR objeto chamado COM:
```

```
        nomeSolicitante
        descricaoProblema
        impacto
        urgencia
        prioridade
      ADICIONAR chamado À lista chamados

      ESCREVER "Chamado cadastrado com sucesso!"
      ESCREVER "Solicitante:", nomeSolicitante
      ESCREVER "Prioridade:", prioridade
    FIM FUNÇÃO

    FUNÇÃO listarChamados()
      SE lista chamados ESTIVER VAZIA ENTÃO
        ESCREVER "Nenhum chamado cadastrado."
      SENÃO
        PARA CADA chamado NA lista chamados FAÇA
          ESCREVER "Nome do solicitante:", chamado.nomeSolicitante
          ESCREVER "Descrição:", chamado.descricaoProblema
          ESCREVER "Impacto:", chamado.impacto
          ESCREVER "Urgência:", chamado.urgencia
          ESCREVER "Prioridade:", chamado.prioridade
        FIM PARA
      FIM SE
    FIM FUNÇÃO

    FUNÇÃO principal()
      continuar <- VERDADEIRO
      ENQUANTO continuar = VERDADEIRO FAÇA
        mostrarMenu()
        ESCREVER "Digite a opção desejada:"
        LER opcao
        ESCOLHA opcao
          CASO 1:
            EXECUTAR cadastrarChamado()
          CASO 2:
            EXECUTAR listarChamados()
          CASO 3:
            ESCREVER "Saindo do Sistema de Triagem de Chamados..."
            continuar <- FALSO
          CASO CONTRÁRIO:
            ESCREVER "Opção inválida. Tente novamente."
        FIM ESCOLHA
      FIM ENQUANTO
    FIM FUNÇÃO

    EXECUTAR principal()
FIM
```

4. Implementação em JavaScript

A implementação do sistema foi desenvolvida em JavaScript. Para facilitar a organização e a compreensão do código, a solução foi estruturada em funções, cada uma responsável por executar uma tarefa específica do processo de triagem dos chamados.

Inicialmente, foi criada a função **mostrarMenu( ),** responsável por apresentar as opções disponíveis no sistema, permitindo ao usuário escolher entre cadastrar um novo chamado, listar os chamados já registrados ou encerrar a aplicação.

Em seguida, a função **classificarPrioridade( )** recebe os valores de impacto e urgência informados durante o cadastro e determina a prioridade correspondente. Essa classificação é realizada por meio de estruturas condicionais (**if, else if e else**), seguindo as regras estabelecidas no enunciado, que define quatro níveis de prioridade: crítica, alta, média e baixa.

A função **cadastrarChamado( )** é responsável por coletar as informações necessárias, como nome do solicitante, descrição do problema, impacto e urgência. Durante essa etapa, também são realizadas validações para garantir que os valores de impacto e urgência estejam dentro das opções permitidas (1, 2 ou 3). Após a validação, a função utiliza **classificarPrioridade( )** para determinar a prioridade do chamado, reúne todas as informações em um objeto e o adiciona ao array **chamados**, responsável por armazenar os registros durante a execução do programa. Ao finalizar o cadastro, o sistema informa ao usuário o nome do solicitante e a prioridade atribuída.

Para a consulta dos registros, foi implementada a função **listarChamados( )**, que verifica inicialmente se existem chamados cadastrados. Caso existam, utiliza uma estrutura de repetição (**for...of**) para percorrer o array e apresentar as informações de cada registro. Caso contrário, exibe uma mensagem informando que nenhum chamado foi cadastrado.

Por fim, a função **principal( )** integra e controla a execução das demais funções. Por meio de uma estrutura de repetição while, mantém o menu disponível até que o usuário escolha encerrar o sistema. A seleção das operações é realizada utilizando a estrutura switch, que direciona a execução para a função correspondente à opção escolhida.

Dessa maneira, as funções trabalham de forma integrada, permitindo que o sistema realize o cadastro, a classificação, o armazenamento e a consulta dos chamados em uma única aplicação. Essa divisão também contribui para a legibilidade, organização e manutenção do código, uma vez que cada funcionalidade possui uma responsabilidade bem definida.

4.1 Testes com diferentes cenários

Teste 01 - Prioridade Crítica.

Dados informados:

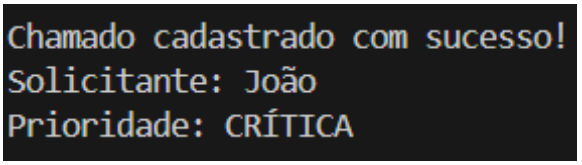
Nome do Solicitante: João

Descrição do Problema: Sistema de vendas indisponível

Impacto: 3

Urgência: 3

Resultado no terminal após cadastro:



Teste 02 - Prioridade Alta.

Dados informados:

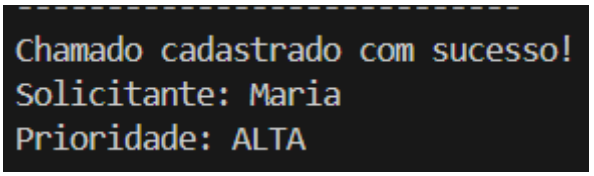
Nome do Solicitante: Maria

Descrição do Problema: Computador do setor financeiro está com tela azul

Impacto: 3

Urgência: 1

Resultado no terminal após cadastro:



Teste 03 - Prioridade Média.

Dados informados:

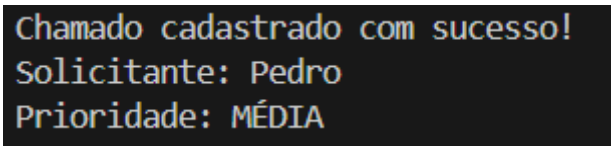
Nome do Solicitante: Pedro

Descrição do Problema: O sistema de cadastro está lento

Impacto: 2

Urgência: 1

Resultado no terminal após cadastro:



Teste 04 - Prioridade Baixa.

Dados informados:

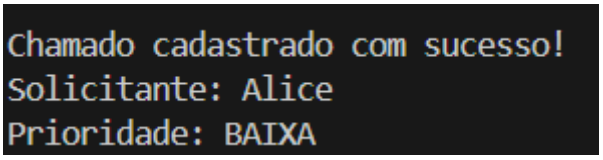
Nome do Solicitante: Alice

Descrição do Problema: O software do meu computador não está atualizado

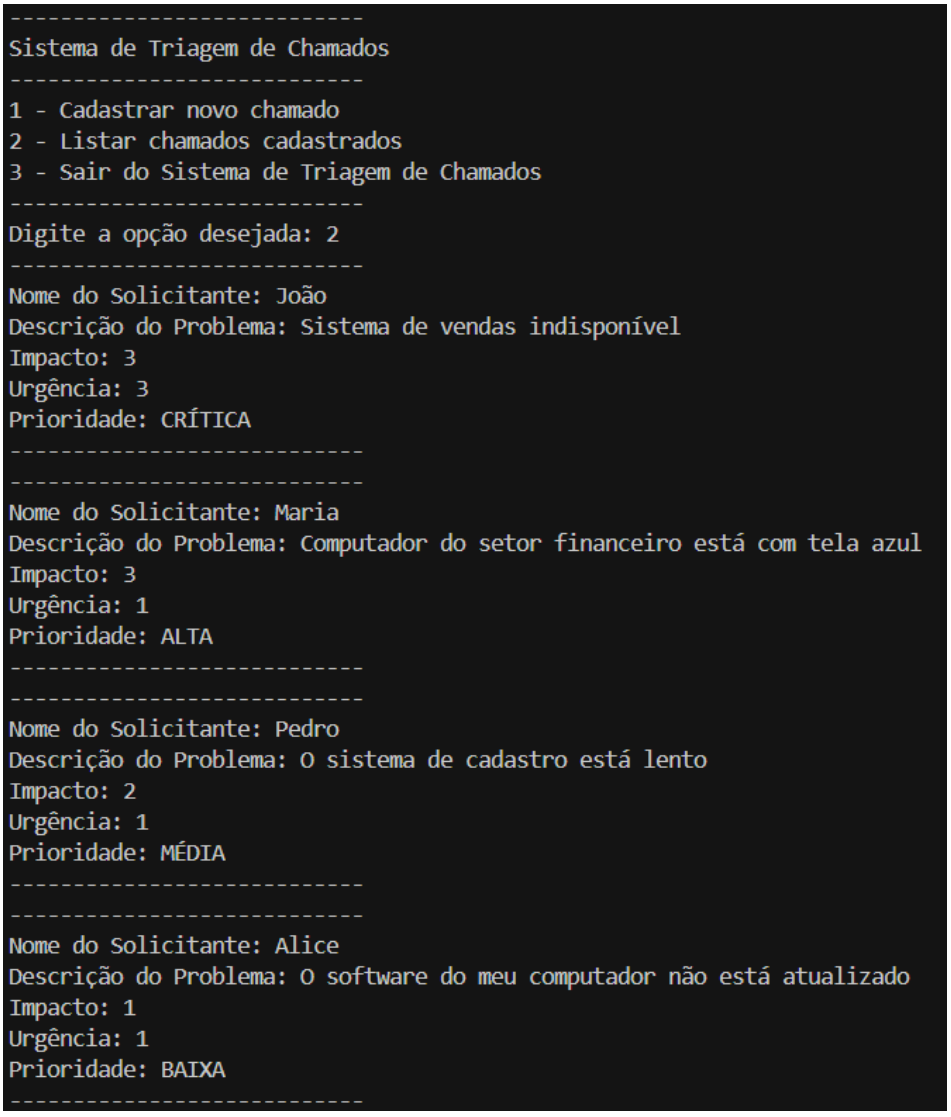
Impacto: 1

Urgência: 1

Resultado no terminal após cadastro:



Após a realização dos cadastros e testes, pode-se selecionar no menu a opção 2 para listar todos os chamados cadastrados no sistema.



4.2 Código

A implementação pode ser conferida no arquivo enviado por meio da pasta compactada, como também pelo seguinte repositório no GitHub: [Triagem_de_Chamados]